

SQL Window Aggregate Functions

15 · Window Aggregations — Complete Reference Guide

Table of Contents

1. COUNT
2. SUM
3. AVG
4. MAX / MIN
5. Rolling Sum & Average Use Case

1. COUNT — Window Aggregation

→ *Task 1: Total orders overall and per customer*

```
SELECT
    OrderID, OrderDate, CustomerID,
    COUNT(*) OVER () AS TotalOrders,
    COUNT(*) OVER (PARTITION BY CustomerID) AS OrdersByCustomers
FROM Sales.Orders
```

→ *Task 2: Count rows, non-null scores, and distinct countries*

```
SELECT
    *,
    COUNT(*) OVER () AS TotalCustomersStar,
    COUNT(1) OVER () AS TotalCustomersOne,
    COUNT(Score) OVER () AS TotalScores,
    COUNT(Country) OVER () AS TotalCountries
FROM Sales.Customers
```

→ *Task 3: Check for duplicate rows in OrdersArchive*

```
SELECT *
FROM (
    SELECT
        *,
```

```

        COUNT(*) OVER (PARTITION BY OrderID) AS CheckDuplicates
    FROM Sales.OrdersArchive
) t
WHERE CheckDuplicates > 1

```

2. SUM — Window Aggregation

→ *Task 4: Total sales overall and per product*

```

SELECT
    OrderID, OrderDate, Sales, ProductID,
    SUM(Sales) OVER () AS TotalSales,
    SUM(Sales) OVER (PARTITION BY ProductID) AS SalesByProduct
FROM Sales.Orders

```

→ *Task 5: Percentage contribution of each product's sales to total*

```

SELECT
    OrderID, ProductID, Sales,
    SUM(Sales) OVER () AS TotalSales,
    ROUND(
        CAST(Sales AS FLOAT) / SUM(Sales) OVER () * 100, 2
    ) AS PercentageOfTotal
FROM Sales.Orders

```

3. AVG — Window Aggregation

→ *Task 6: Average sales overall and per product*

```

SELECT
    OrderID, OrderDate, Sales, ProductID,
    AVG(Sales) OVER () AS AvgSales,
    AVG(Sales) OVER (PARTITION BY ProductID) AS AvgSalesByProduct
FROM Sales.Orders

```

→ *Task 7: Average customer scores — with and without NULLs*

```

SELECT
    CustomerID, LastName, Score,
    COALESCE(Score, 0) AS CustomerScore,
    AVG(Score) OVER () AS AvgScore,
    AVG(COALESCE(Score, 0)) OVER () AS AvgScoreWithoutNull
FROM Sales.Customers

```

→ *Task 8: Orders where Sales exceed the overall average*

```

SELECT *
FROM (
    SELECT
        OrderID, ProductID, Sales,
        AVG(Sales) OVER () AS Avg_Sales

```

```
FROM Sales.Orders
) t
WHERE Sales > Avg_Sales
```

4. MAX / MIN — Window Aggregation

→ Task 9: Highest and lowest sales (standard aggregate)

```
SELECT
    MIN(Sales) AS MinSales,
    MAX(Sales) AS MaxSales
FROM Sales.Orders
```

→ Task 10: Lowest sales overall and per product

```
SELECT
    OrderID, ProductID, OrderDate, Sales,
    MIN(Sales) OVER () AS LowestSales,
    MIN(Sales) OVER (PARTITION BY ProductID) AS LowestSalesByProduct
FROM Sales.Orders
```

→ Task 11: Show employees with the highest salary

```
SELECT *
FROM (
    SELECT
        *,
        MAX(Salary) OVER () AS HighestSalary
    FROM Sales.Employees
) t
WHERE Salary = HighestSalary
```

→ Task 12: Deviation of each sale from min and max sales

```
SELECT
    OrderID, OrderDate, ProductID, Sales,
    MAX(Sales) OVER () AS HighestSales,
    MIN(Sales) OVER () AS LowestSales,
    Sales - MIN(Sales) OVER () AS DeviationFromMin,
    MAX(Sales) OVER () - Sales AS DeviationFromMax
FROM Sales.Orders
```

5. Rolling Sum & Average — Use Case

→ Task 13: Moving average of sales per product over time

```
SELECT
    OrderID, ProductID, OrderDate, Sales,
    AVG(Sales) OVER (PARTITION BY ProductID)           AS AvgByProduct,
    AVG(Sales) OVER (PARTITION BY ProductID
                     ORDER BY OrderDate)              AS MovingAvg
```

```
FROM Sales.Orders
```

→ *Task 14: Rolling average including only current and next order*

```
SELECT
    OrderID, ProductID, OrderDate, Sales,
    AVG(Sales) OVER (
        PARTITION BY ProductID
        ORDER BY OrderDate
        ROWS BETWEEN CURRENT ROW AND 1 FOLLOWING
    ) AS RollingAvg
FROM Sales.Orders
```